



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Microcontroladores.

Práctica No. 3: Uso de Puertos y Retardos por Software en el ATmega2560.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Jesús A. García L.

Agosto/2026

Tabla de contenido

I.	Objetivo	3
II.	Descripción de la practica.....	3
	Lista de materiales.....	3
III.	Desarrollo de la práctica	3
	Teoría.....	3
	Retardos por software: conteo de ciclos de instrucción	3
	Generación de números pseudoaleatorios: LFSR	4
	Análisis matemático de los retardos.....	4
	Retardo de 52 μ s	4
	Retardo de 16 ms	5
	Retardo de 1 s.....	6
	Código fuente (Prac3.asm).....	7
	Explicación de myRand línea por línea	10
	Resultados de simulación	11
	Reflexión y análisis crítico	12
	Dificultades técnicas.....	12
	Fracasos, fallas, bugs y errores.....	12
	Cómo se resolvió.....	12
	Conclusión	12
IV.	Bibliografía.....	12

I. Objetivo

El alumno aprenderá la forma básica de implementar retardos por software mediante el conteo de ciclos de instrucción del microcontrolador, calculando de manera precisa los contadores necesarios para producir retardos de 52 μ s, 16 ms y 1 s en un ATmega2560 a 16 MHz. Adicionalmente, implementará un procedimiento que genere números pseudoaleatorios de 8 bits mediante un registro de desplazamiento con retroalimentación lineal (LFSR).

II. Descripción de la practica

Crear un proyecto de ensamblador en AVR Studio para el dispositivo ATmega2560, partiendo del Listado 1 (Prac3.asm) que ya contiene la inicialización de R24 y el ciclo principal de conteo.

A partir de la tabla de ciclos por instrucción del AVR Instruction Set Manual (DS40002198), deducir la fórmula general del número de ciclos de un lazo de retardo, tanto simple como anidado.

Calcular los valores exactos de los contadores necesarios para retardos de 52 μ s, 16 ms y 1 s a 16 MHz, e implementar las subrutinas delay52uS, delay16mS y delay1S.

Implementar la subrutina myRand, que genera un número pseudoaleatorio de 8 bits mediante un LFSR a partir de una semilla cargada en R24, devolviendo el resultado en R25.

Compilar el proyecto y, con breakpoints en el simulador de AVR Studio, verificar que cada retardo tarda el tiempo esperado y que myRand recorre valores distintos en cada llamada.

Redactar las conclusiones y comentarios de la práctica, documentando el desarrollo matemático y las dificultades encontradas.

Lista de materiales

- Computadora personal (PC).
- AVR Studio (IDE de Atmel/Microchip).
- Datasheet del ATmega2560 / AVR Instruction Set Manual DS40002198 (tabla de ciclos por instrucción).
- Editor de texto y terminal con acceso a git.

III. Desarrollo de la práctica

Teoría

Retardos por software: conteo de ciclos de instrucción

En el núcleo clásico del AVR, casi todas las instrucciones ejecutan en un número fijo de ciclos de reloj, tabulado en la Sección 5 ("Instruction Set Summary") del AVR Instruction Set Manual. Con $F_{CPU} = 16 \text{ MHz}$, cada ciclo dura $T = 1/F_{CPU}$

UABC_FCQI_MICROCONTROLADORES.

= 62.5 ns, y el tiempo total de una rutina es la suma de los ciclos de las instrucciones que realmente se ejecutan, multiplicada por T.

Para el ATmega2560 (PC de 17 bits, por sus 128K palabras de flash) los ciclos relevantes son: NOP, LDI, DEC, INC, CPI, OUT y MOV = 1 ciclo; BRNE/BREQ = 1 ciclo si no salta (última vuelta del lazo) o 2 ciclos si salta; RJMP = 2 ciclos; y CALL/RET = 5 ciclos cada uno (en el ATmega1280, con PC de 16 bits, CALL/RET cuestan 4 ciclos).

Un lazo de retardo básico (f NOPs de relleno + DEC + BRNE, repetido N veces) cuesta Ciclos = $N \cdot (f+3) - 1$, donde el -1 corresponde a que en la última vuelta el BRNE ya no salta y ahorra un ciclo. Anidando un nivel más (contador externo M, interno N), el costo es Ciclos = $M \cdot [f_1 + N \cdot (f_2+3) + 3] - 1$, y así sucesivamente para más niveles. Sumando el CALL, el LDI inicial y el RET de la subrutina, el problema de diseño se reduce a despejar los enteros N, M, ... (acotados a 256 por ser registros de 8 bits) que hacen que la fórmula sea exactamente igual a Ciclos_objetivo = $T_{deseado} \cdot F_{CPU}$.

Generación de números pseudoaleatorios: LFSR

Un microcontrolador no tiene una fuente física de entropía, así que un “número aleatorio” generado por software es en realidad pseudoaleatorio: una secuencia determinista que, a partir de una semilla, parece desordenada pero es completamente reproducible. El método estándar para esto es el LFSR (Linear Feedback Shift Register): un registro de desplazamiento cuyo bit de entrada es el XOR de ciertos bits del estado actual (los “taps”).

Los taps definen un polinomio característico sobre GF(2). Con taps en los bits 7, 5, 4 y 3 (0-indexados), el polinomio es $x^8+x^6+x^5+x^4+1$, que es primitivo: garantiza que el LFSR recorra las $2^8-1 = 255$ combinaciones distintas de 8 bits antes de repetirse (periodo máximo). La semilla nunca puede ser 0x00, porque en ese estado todos los taps son 0, el bit de retroalimentación siempre es 0, y el LFSR queda atascado en 0 para siempre (es un punto fijo, no parte de la secuencia).

En software, la regla $\text{bit_nuevo} = b_7 \oplus b_5 \oplus b_4 \oplus b_3$ seguida de $\text{registro} \leftarrow (\text{registro} \ll 1) \mid \text{bit_nuevo}$ se implementa extrayendo cada bit con BST/BLD, acumulando el XOR, y usando LSR (para sacar el bit de retroalimentación hacia el Carry) seguido de ROL (para desplazar e insertar ese Carry) en una sola instrucción.

Análisis matemático de los retardos

Retardo de 52 μs

Ciclos objetivo: $C = 52 \times 10^{-6} \text{ s} \times 16 \times 10^6 \text{ Hz} = 832$ ciclos. Con un solo lazo: $832 = \text{CALL} + \text{LDI} + [N(f+3)-1] + \text{RET} = N(f+3) + 10$, de donde $N(f+3) = 822 = 2 \times 3 \times 137$. De los divisores de 822, la única combinación con $N \leq 256$ y $f \geq 0$ razonable es $N=137$, $f+3=6$ ($f=3$). Verificación: $137(3+3)-1 = 821$; $821+1+5+5 = 832$ ciclos $\rightarrow 832/16 \text{ MHz} = 52 \mu\text{s}$ exacto.

UABC_FCQI_MICROCONTROLADORES.

```

;----- definiciones e includes -----
INCLUDE "m2560def.inc"      ; Incluir definiciones de Registros para 2560

.EQU INIT_VALUE = 0         ; Valor inicial R24
.EQU SEED_VALUE = 0xA5      ; Semilla inicial para myRand (nunca debe ser 0x00)

;----- inicializar -----
LDI R24, INIT_VALUE
;----- implementar -----
CALL delay52uS
CALL delay16mS
CALL delay1S
CALL myRand                 ; Retorna valor en R25

NOP

;----- ciclo principal -----
arriba: INC R24
CPI R24, 10
BRREQ abajo
OUT PORTA, R24
RAMP arriba

abajo: DEC R24
CPI R24, 0
BRREQ arriba
OUT PORTA, R24
RAMP abajo

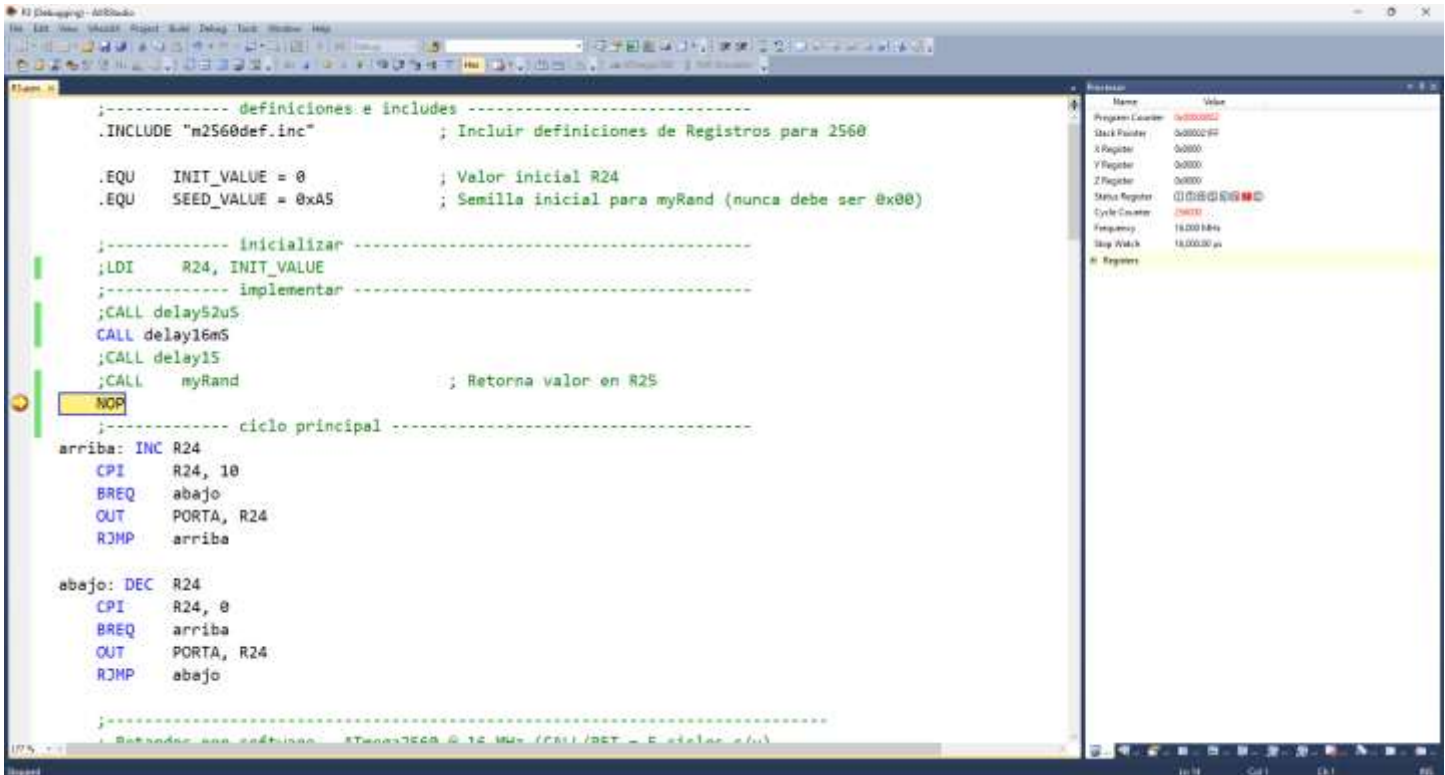
;----- Retardos por software - ATmega2560 @ 16 MHz (CALL/RET = 5 ciclos c/u) -----
; delay52uS: 52 us -> Ciclos = 137*(3+3)+10 = 832 ; 832/16MHz = 52us exacto
; N*(f_3+3) + c + 10
delay52uS:

```

Register	Value
Program Counter	0x00000022
Stack Pointer	0x000000FF
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	0x00000000
Cycle Counter	0x00000000
Frequency	16.000 MHz
Stop Watch	00.00 us

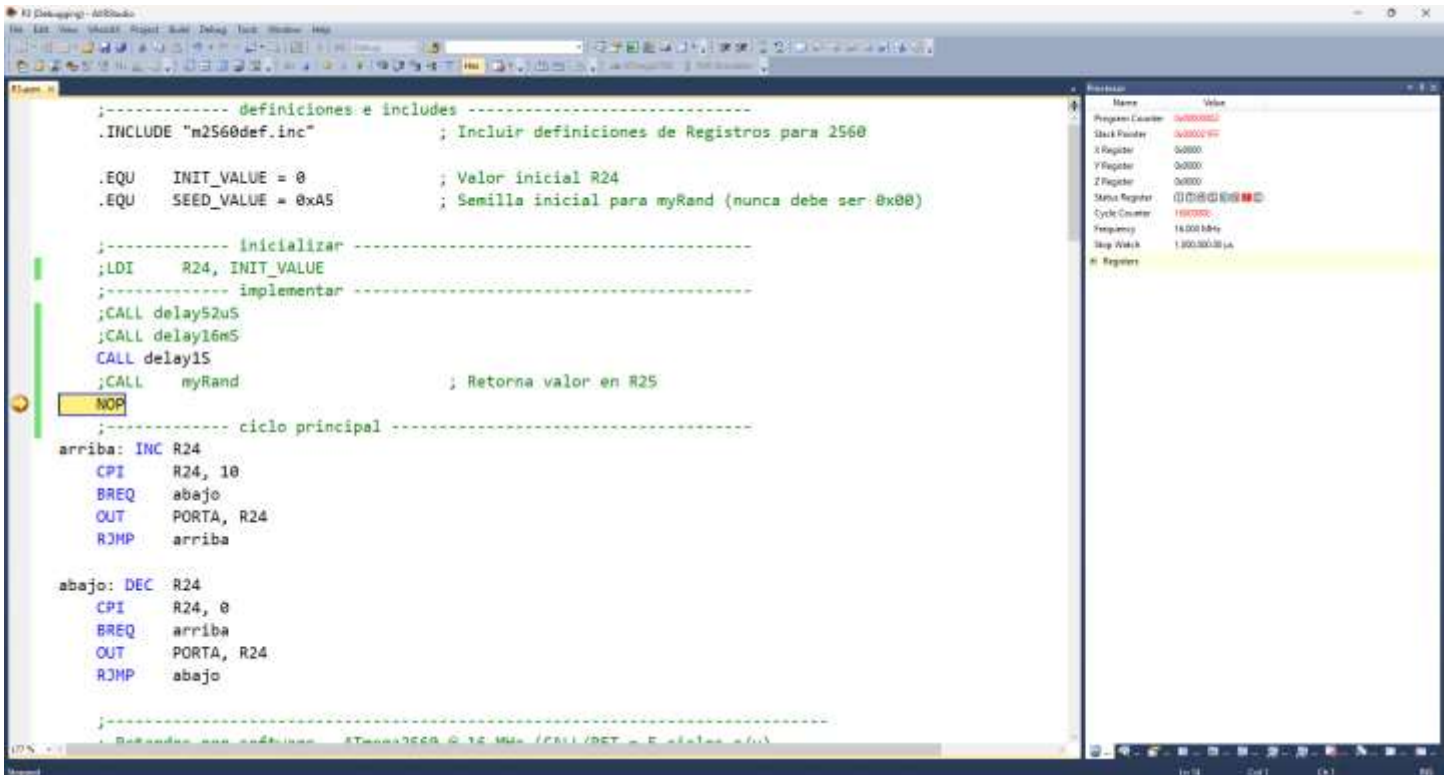
Retardo de 16 ms

Ciclos objetivo: $C = 16 \times 10^{-3} \times 16 \times 10^6 = 256\,000$ ciclos. Con lazo anidado doble: $256\,000 = 11 + M[f_1 + N(f_2 + 3) + 3] - 1$, es decir $M[f_1 + N(f_2 + 3) + 3] = 255\,990 = 2 \times 3 \times 5 \times 7 \times 23 \times 53$. Tomando $f_1 = 0$ y $f_2 = 2$ ($N(f_2 + 3) = 5N$), se busca un divisor $M \leq 256$ de $255\,990$ tal que $(255\,990/M - 3)/5$ sea entero ≤ 256 : $M = 230$ da $255\,990/230 = 1113$, $(1113 - 3)/5 = 222 = N$. Verificación: lazo interno = $222 \times 5 - 1 = 1109$; total = $230 \times 1113 - 1 + 1 + 11 = 256\,000$ ciclos $\rightarrow 16$ ms exacto.



Retardo de 1 s

Ciclos objetivo: $C = 1 \times 16 \times 10^6 = 16\,000\,000$ ciclos. Con tres incógnitas enteras acopladas ($P, M, N \leq 256$) más los rellenos g, f_1, f_2 , el espacio de combinaciones es demasiado grande para factorizar a mano, así que se resolvió con una búsqueda sistemática de divisores (el mismo principio de los dos casos anteriores, aplicado computacionalmente). Solución exacta encontrada: $P=241, M=187, N=117, g=2, f_1=1, f_2=0$ (sin NOPs sueltos de relleno). Verificación: lazo interno = $117 \times 3 - 1 = 350$; nivel medio = $187 \times 350 - 1 = 66\,384$; nivel externo = $241 \times 66\,390 - 1 = 15\,999\,989$; $+1+11 = 16\,000\,000$ ciclos $\rightarrow$ exactamente 1 s.



```

;----- definiciones e includes -----
.INCLUDE "m2560def.inc"           ; Incluir definiciones de Registros para 2560

.EQU  INIT_VALUE = 0              ; Valor inicial R24
.EQU  SEED_VALUE = 0xA5           ; Semilla inicial para myRand (nunca debe ser 0x00)

;----- inicializar -----
;LDI R24, INIT_VALUE
;----- implementar -----
;CALL delay52uS
;CALL delay16mS
CALL delay1S
;CALL myRand                      ; Retorna valor en R25
NOP
;----- ciclo principal -----
arriba: INC R24
CPI R24, 10
BREQ abajo
OUT PORTA, R24
RJMP arriba

abajo: DEC R24
CPI R24, 0
BREQ arriba
OUT PORTA, R24
RJMP abajo

;-----
; Defining some constants: ATmega2560 @ 16 MHz (F_CPU) (DEF = E = 16000000)

```

Name	Value
Program Counter	0x00000001
Stack Pointer	0x00000000
X Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	0x00000000
Cycle Counter	16000000
Frequency	16,000 MHz
Stop Watch	1,000,000.00 µs
R Registers	

Código fuente (Prac3.asm)

El proyecto se creó para el dispositivo ATmega2560. El archivo fuente, con las tres subrutinas de retardo y myRand ya integradas, quedó como sigue:

UABC_FCQI_MICROCONTROLADORES.

```

;----- definiciones e includes -----
.INCLUDE "m2560def.inc" ; Incluir definiciones de Registros para 2560

.equ INIT_VALUE = 0 ; Valor inicial R24
.equ SEED_VALUE = 0xA5 ; Semilla inicial para myRand (nunca debe ser 0x00)

;----- inicializar -----
ldi R24,INIT_VALUE
;----- implementar -----
;CALL delay51uS
;CALL delay52uS
;CALL delay16mS
;CALL delay1S
;CALL myRand ; Retorna valor en R25
;----- ciclo principal -----
arriba: inc R24
        cpi R24,10
        breq abajo
        out PORTA,R24
        rjmp arriba

abajo: dec R24
        cpi R24,0
        breq arriba
        out PORTA,R24
        rjmp abajo

;-----
; Retardos por software - ATmega2560 @ 16 MHz (CALL/RET = 5 ciclos c/u)
;-----

; delay51uS: 52 us -> Ciclos = 137*(3+3)+10 = 832 ; 832/16MHz = 52us exacto
; N*(f_1+3) + c + 10
delay52uS:
        ldi R16, 137 ; N
        ; c
        _d52u_L1_loop:
            ; f_1
            nop
            nop
            nop
            dec R16
            brne _d52u_L1_loop
ret

```

UABC_FCQI_MICROCONTROLADORES.

```
; delay16mS: 16 ms -> Ciclos = 230*[222*6+3]+10 = 256000 ; 256000/16MHz = 16ms exacto
; Ciclos = M[f_2 + N{f_1 + 3} + 3] + c + 10
```

```
delay16mS:
```

```
    ldi    R17, 230        ; M
    ;c
    _d16ms_L2_loop:
        ldi    R16, 222    ; N
        ;f_2
        _d16ms_L1_loop:
            ;f_1
            nop
            nop
            dec    R16
            brne  _d16ms_L1_loop
        dec    R17
        brne  _d16ms_L2_loop
```

```
ret
```

```
; delay1S: 1 s -> Ciclos = 16,000,000 exactos ; 16000000/16MHz = 1s exacto
; Ciclos = P(f_3 + M[f_2 + N{f_1 + 3} + 3] + 3) + c + 10
```

```
delay1S:
```

```
    ldi    R19, 241        ; P
    ; c
    _d1s_L3_loop:
        ; f_3
        nop
        nop
        ldi    R20, 187    ; M
        _d1s_L2_loop:
            ; f_2
            nop
            ldi    R21, 117    ; N
            _d1s_L1_loop:
                ; f_1
                dec    R21
                brne  _d1s_L1_loop
            dec    R20
            brne  _d1s_L2_loop
        dec    R19
        brne  _d1s_L3_loop
```

```
ret
```

```

;-----
; myRand: numero pseudoaleatorio de 8 bits (LFSR, polinomio x^8+x^6+x^5+x^4+1)
; Entrada: R24 (semilla)      Salida: R25 (siguiente valor del LFSR)
; R24 no se modifica; R16 y R17 se usan como auxiliares internos.
;-----
myRand:
    clr    R17          ; acumulador del bit de retroalimentacion (bit0)

    bst    R24, 7       ; T = bit7 de la semilla
    clr    R16
    bld    R16, 0
    eor    R17, R16

    bst    R24, 5       ; T = bit5
    clr    R16
    bld    R16, 0
    eor    R17, R16

    bst    R24, 4       ; T = bit4
    clr    R16
    bld    R16, 0
    eor    R17, R16

    bst    R24, 3       ; T = bit3
    clr    R16
    bld    R16, 0
    eor    R17, R16

    mov    R25, R24     ; copiamos la semilla a R25 para desplazarla ahi
    lsr    R17          ; Carry = bit de retroalimentacion
    rol    R25          ; R25 = (R25<<1) | Carry -> nuevo estado del LFSR
    ret

```

Explicación de myRand línea por línea

- `clr R17`: pone en 0 el acumulador donde se va armando el XOR de los taps (XOR con 0 es neutro).
- `bst R24,7 / clr R16 / bld R16,0 / eor R17,R16`: copia el bit 7 de la semilla al flag T (bst), lo traslada al bit 0 de un registro limpio (clr+bld), y lo acumula en R17 con XOR. El mismo patrón se repite para los bits 5, 4 y 3, de modo que al terminar, el bit 0 de R17 contiene $b_7 \oplus b_5 \oplus b_4 \oplus b_3$.
- `mov R25,R24`: copia la semilla a R25, que es donde se va a desplazar, dejando R24 intacto para quien haya llamado a la subrutina.
- `lsr R17`: desplaza R17 a la derecha; el bit 0 (la retroalimentación ya calculada) cae en el flag Carry.
- `rol R25`: desplaza R25 a la izquierda insertando el Carry en el bit 0 — en una sola instrucción hace el $(\text{registro} \ll 1) \mid \text{bit_nuevo}$ de la teoría.
- `ret`: regresa con el nuevo número pseudoaleatorio en R25, listo para usarse o para alimentar la siguiente llamada.

Resultados de simulación

```

;----- definiciones e includes -----
.INCLUDE "m2560def.inc"           ; Incluir definiciones de Registros para 2560

.EQU  INIT_VALUE = 0              ; Valor inicial R24
.EQU  SEED_VALUE = 0xA5           ; Semilla inicial para myRand (nunca debe ser 0x0)

;----- inicializar -----
;LDI R24, INIT_VALUE
LDI R24, SEED_VALUE
;----- implementar -----
;CALL delay52uS
;CALL delay16mS
;CALL delay1S
CALL myRand                       ; Retorna valor en R25
NOP

;----- ciclo principal -----
arriba: INC R24
CPI R24, 10
BREQ abajo
OUT PORTA, R24
RJMP arriba

abajo: DEC R24
CPI R24, 0
BREQ arriba
OUT PORTA, R24
RJMP abajo

```

Name	Value
Program Counter	0x00000000
Stack Pointer	0x000010FF
R Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	0x00000000
Cycle Counter	0
Frequency	16.000 MHz
Stop Watch	0.00 µs
R Register	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x00
R21	0x00
R22	0x00
R23	0x00
R24	0x45
R25	0x00
R26	0x00
R27	0x00
R28	0x00
R29	0x00

```

;----- definiciones e includes -----
.INCLUDE "m2560def.inc"           ; Incluir definiciones de Registros para 2560

.EQU  INIT_VALUE = 0              ; Valor inicial R24
.EQU  SEED_VALUE = 0xA5           ; Semilla inicial para myRand (nunca debe ser 0x0)

;----- inicializar -----
;LDI R24, INIT_VALUE
LDI R24, SEED_VALUE
;----- implementar -----
;CALL delay52uS
;CALL delay16mS
;CALL delay1S
CALL myRand                       ; Retorna valor en R25
NOP

;----- ciclo principal -----
arriba: INC R24
CPI R24, 10
BREQ abajo
OUT PORTA, R24
RJMP arriba

abajo: DEC R24
CPI R24, 0
BREQ arriba
OUT PORTA, R24
RJMP abajo

```

Name	Value
Program Counter	0x00000000
Stack Pointer	0x000010FF
R Register	0x0000
Y Register	0x0000
Z Register	0x0000
Status Register	0x00000000
Cycle Counter	11
Frequency	16.000 MHz
Stop Watch	1.00 µs
R Register	
R00	0x00
R01	0x00
R02	0x00
R03	0x00
R04	0x00
R05	0x00
R06	0x00
R07	0x00
R08	0x00
R09	0x00
R10	0x00
R11	0x00
R12	0x00
R13	0x00
R14	0x00
R15	0x00
R16	0x00
R17	0x00
R18	0x00
R19	0x00
R20	0x00
R21	0x00
R22	0x00
R23	0x00
R24	0x45
R25	0x00
R26	0x00
R27	0x00
R28	0x00
R29	0x00

Para cada subrutina se recomienda descomentar un único CALL a la vez, colocar un breakpoint justo después del RET, y usar el contador de ciclos del simulador (Processor Status / Cycle Counter) para confirmar que delay52uS, delay16mS y

delay1S consumen 832, 256 000 y 16 000 000 ciclos respectivamente. Para myRand, se recomienda cargar distintos valores en R24 antes de cada llamada y verificar que R25 cambia de forma no trivial, y que la semilla 0x00 efectivamente produce siempre 0x00 en R25 (el punto fijo previsto por la teoría).

Reflexión y análisis crítico

Dificultades técnicas

La plantilla original del profesor asumía un ATmega1280 (CALL/RET = 4 ciclos), mientras que el hardware realmente usado es un ATmega2560 (CALL/RET = 5 ciclos, por su PC de 17 bits). Además, el retardo de 1 s requiere un lazo triple anidado cuyo espacio de combinaciones enteras ($P, M, N \leq 256$, más los NOPs de relleno) es demasiado grande para resolverse por factorización manual.

Fracasos, fallas, bugs y errores

Los primeros intentos de retardo (un lazo de 195×15 con NOPs sueltos) fueron exploratorios y no correspondían a ningún tiempo objetivo exacto — simplemente se habían elegido valores arbitrarios sin partir de la fórmula de ciclos. De igual forma, los primeros intentos de armar el lazo triple para 1 s a mano, con relleno de NOPs elegido por prueba y error, no cuadraban exactamente con los 16 000 000 de ciclos requeridos.

Cómo se resolvió

Se dedujo la fórmula general de ciclos de un lazo de retardo (simple y anidado) a partir de la tabla de ciclos por instrucción del AVR Instruction Set Manual, y se resolvió la búsqueda de contadores enteros exactos mediante una búsqueda sistemática de divisores del número de ciclos objetivo, encontrando combinaciones exactas de N, M, P y NOPs de relleno sin necesidad de aproximaciones ni redondeos.

Conclusión

Esta práctica permitió entender que un retardo por software no es más que aritmética de ciclos: una vez tabulado el costo de cada instrucción, calcular un tiempo exacto se reduce a resolver una ecuación entera. El mismo principio de estado determinista y transformación reproducible aparece en el LFSR: un mecanismo simple (un XOR y un corrimiento) que, elegido con el polinomio correcto, produce una secuencia con propiedades estadísticas útiles pese a no tener nada de “aleatorio” en el sentido físico.

IV. Bibliografía

- Microchip Technology / Atmel. (2021). AVR® Instruction Set Manual, DS40002198.
- Microchip Technology / Atmel. ATmega640/V-1280/V-1281/V-2560/V-2561/V Datasheet, DS40002211A.
- Alfke, P. (1996). Efficient Shift Registers, LFSR Counters, and Long Pseudo-Random Sequence Generators. Xilinx Application Note XAPP 052.
- Universidad Autónoma de Baja California, Facultad de Ciencias Químicas e Ingeniería. (2026). Práctica 3: Uso de Puertos y retardos mediante Software.